

Honour Code

1. You may not use generative AI to create any part of your solution (e.g. ChatGPT).
2. Do not copy the answers from any source.
3. You may not receive or share any part of your code with anyone.
4. Cheating students will face the Committee on Discipline.

Important Note:

This assignment is handled by Jon Simon, who does not speak Hebrew. Therefore, any questions about this assignment should be posted to Piazza in English to ensure you get a response.

Assignment Details

Gott Taxi encourages Drivers to get as many five-star reviews from Riders as possible.

Among Drivers, there are two competing theories about the best way to get five-star reviews.

Some Drivers believe that Riders who give the *highest reviews on average* are more likely to give them a good review, they believe these Riders are easily impressed and probably give five-star ratings often. These Drivers will always pick up the Rider who gives the highest average review.

Other Drivers believe that Riders who give the *lowest reviews on average* are more likely to give them a good review, they believe these Riders have had such bad experiences in the past that they will be easily impressed by good Drivers. These drivers will always pick up the Rider who gives the lowest average review.

Since *Gott Taxi* only cares about getting five-star reviews, they are happy to support both types of Driver, letting each Driver choose the Rider who gives either the highest or lowest average review (whichever the Driver prefers).

In this assignment you will implement a general-purpose heap data structure which acts as both a min-heap and max-heap simultaneously. The heap will be used to help *Gott Taxi* assign Riders to Drivers. The Riders in the heap are currently waiting to be picked up. When a Driver wants to pick up a Rider they choose one from the heap.

The project consists of two Java classes and some Java Test classes.

1. `GottRider.java` represents a single Rider. Each Object stores details about the customer such as their name, number of previous rides, average review they have given, etc..
2. `MinMaxHeap.java` is not specific to *Gott Taxi*. It is a *generic* heap capable of storing any Object with one restriction: the heap can only store *Comparable* objects. A basic understanding of Generics in Java will be necessary. `GottRider.java` is a *Comparable* class since it implements the *Comparable* interface. This means the `MinMaxHeap` will be able to store those objects.

3. The test classes

We have provided tests for your classes, they do not cover all cases (they cover very few edge cases). It is vital that you add (many) more tests for your code. The test files include guidance on how to write tests, and you can look at the provided tests for inspiration. We also included some functions that will be useful for testing your code.

`BasicGottRiderTest.java` is basically complete, but you can add tests if you wish.

`BasicMinMaxHeapTest.java` is where you should add your tests for the `MinMaxHeap`.

Be certain you have tests checking all the behaviours you expect `MinMaxHeap` to exhibit.

Create Project (we recommend IntelliJ IDEA)

IntelliJ IDEA can be downloaded freely on (almost) any platform, we recommend using it for this homework. A demonstration of how to set up the project in IntelliJ is available as a video on the piazza in the post announcing this homework. The recording includes a comprehensive guide on setting up your project, including forking the repository in INGINious and creating the project in IntelliJ. We **strongly** recommend you watch the recording.

The Java files you write should be placed in the (currently empty) folder `src/main/java`.

IntelliJ gives you the ability to run the tests we provided, and can help you write cleaner code, detect bugs, etc.. This is a good opportunity to get familiar with its features!

Class Descriptions

A full list of each class's required public methods can be found at the end of this PDF.

This section lists constraints/requirements which your code must follow *exactly*.

We intentionally do not provide you with skeleton Java files for these classes because it is important that you gain experience with writing a project from scratch. Some smaller details are not fully explained, we encourage you to use your best judgement and make decisions you can justify. These are not details/decisions that will affect the correctness of your code.

GottRider (public class GottRider implements Comparable<GottRider>)

This class implements the interface `Comparable<GottRider>`. This means it is required to have a function `compareTo` which takes another `GottRider` as a parameter, and determines which of the two `GottRiders` is "less than" the other. Rider *A* is "less than" Rider *B* if *A* gives lower average reviews than rider *B*. If you are not familiar with how the return value of `compareTo` represents which object is smaller, you should read about "Comparable" in Java before implementing that part of the code.

The class should have several fields (you can add more if you require them):

- `name` (a String)
- `previousRides` (a Long)
- `averageReview` (a Double)

Public Methods

`GottRider` contains only simple methods without much logic.

See the code listing at the end of this PDF for details and instructions.

MinMaxHeap (public class MinMaxHeap<T extends Comparable<T>>)

As you can see in the code listing towards the end of this PDF, `MinMaxHeap` has three constructors for you to implement. They each serve a different purpose to users of `MinMaxHeap`.

- Constructor taking no parameters. Creates a heap with a sensible default capacity (“sensible” is subjective; use your best judgement).
- Constructor taking a single parameter, an int `initialCapacity` which defines the initial number of elements the heap can store before needing to resize.
- Constructor taking a single parameter, an array `initialData`. If you use `T` as the type variable of `MinMaxHeap` then the type of this parameter should be `T[]`. (If this description doesn’t make sense to you, consider reading about *Generic Types* in Java). This constructor **must** run in $O(n)$ time where n is the length of the given array.

Public Methods

`insert` $O(\log n)$ if heap is not full, $O(n)$ otherwise: Add an element to the heap.

Inserting into a **full heap** should not result in an error/exception. The Heap should **double its capacity** to hold more elements, then insert the element as usual.

`deleteMin` $O(\log n)$: Remove the minimum element from the heap and return it.

Calling this method on an empty heap must throw an Exception.

`findMin` $O(1)$: Return the minimum element in the heap.

Calling this method on an empty heap must throw an Exception.

`deleteMax` $O(\log n)$: Remove the maximum element from the heap and return it.

Calling this method on an empty heap must throw an Exception.

`findMax` $O(1)$: Return the maximum element in the heap.

Calling this method on an empty heap must throw an Exception.

`getSize` $O(1)$: Return the number of elements in the heap.

Implementation Guidelines

`MinMaxHeap` should maintain both a min-heap and a max-heap (both implemented as arrays). Each element in the `MinMaxHeap` will be stored in both the min-heap and the max-heap. You should use two additional arrays to map the index of elements from each heap to the other, this will allow you to keep track of where any element from the min-heap is stored in the max-heap without needing to search the entire max-heap (and vice-versa). This is necessary if we want to support find/delete in $O(\log n)$, try to think *why* as it will help you understand `MinMaxHeap`.

To be overly precise, if the array representing the min-heap is H_1 , the array representing the max-heap is H_2 , the array mapping elements from H_1 to H_2 is $P_{1 \rightarrow 2}$, and the array mapping elements from H_2 to H_1 is $P_{2 \rightarrow 1}$ then for all indices i :

$$H_1[i] = H_2[P_{1 \rightarrow 2}[i]]$$

and

$$H_2[i] = H_1[P_{2 \rightarrow 1}[i]]$$

I.e., to find where an element at position i in H_1 (min-heap) is located in H_2 (max-heap), you need only check $P_{1 \rightarrow 2}[i]$, and likewise for finding where an element in H_2 is located in H_1 .

Remember: **all four arrays** must be updated whenever an element is added/deleted/moved/etc. *and* whenever the heap is resized.

Misc. Guidelines

Note: All running times listed refer to the *worst-case* running time.

You may need to read about *Generic Types*, the *Comparable* interface, and some other topics of Java. Generics and Interfaces are used in (almost) every non-trivial Java codebase, so it's worth understanding them well.

You are allowed to:

- Add new private functions to help you (and to keep the code clean and readable).
- Add new private members/fields to classes if you find it convenient.

You are not allowed to:

- Remove or modify the signature of public functions in any class.
- Rename classes or public members/fields.
- Import anything unnecessary that might defeat the purpose of implementing a heap, the official solution has no imports.

You will need to add some fields we do not mention in order to support certain functions of the classes. This is both necessary and encouraged.

Submission instructions

The assignment must be submitted individually, **not in pairs**

A demonstration of how to submit your code is available as a video on the piazza. You will fork the assignment's repository in INGINious. Submissions will also be handled in INGINious. When you submit, some tests are run automatically and you receive a score. **This is not your grade.** The grader will run additional tests which you cannot access. Your grade will be based on *all* tests.

Take the time to go over your code and ensure it is understandable, not excessively long, and well formatted. With a little thought, implementations can be kept reasonably short.

Ensure that your code is well documented (according to the standards you saw in Introduction to Computer Science).

Deviations from these guidelines may result in a point penalty.

Deadline

Submit no later than the 1st of May. You may use an automatic extension up to the 5th of May.

Function Signatures and Time Complexities

Listing 1: MinMaxHeap's public methods/constructors

```
1 // O(1) (In this course, creating an empty array of size n is O(1))
2 public MinMaxHeap();
3
4 // O(1) (In this course, creating an empty array of size n is O(1))
5 public MinMaxHeap(int initialCapacity);
6
7 // O(n) where n is the size of the initial data
8 public MinMaxHeap(T[] initialData);
9
10 // O(log n) if heap is not full, otherwise O(n) where n is the size of the heap
11 public void insert(T element);
12
13 // O(log n) where n is the size of the heap
14 public T deleteMin();
15
16 // O(log n) where n is the size of the heap
17 public T deleteMax();
18
19 // O(1)
20 public T findMin();
21
22 // O(1)
23 public T findMax();
24
25 // O(1)
26 public int getSize();
```

Listing 2: GottRider's public methods/constructors

```
1 // O(1) - Constructor creating a new GottRider with the provided information
2 public GottRider(String name, Long previousRides, Double averageReviewGiven);
3
4 // O(1) - Called when GottRider completes a ride, updates averageReviewGiven
5 public void rideCompleted(Long starsGiven);
6
7 // O(1) - Get name passed to constructor
8 public String getName();
9
10 // O(1) - Returns a sensible representation of this GottRider.
11 public String toString();
12
13 // O(1) - We suggest you read about Java's Comparable interface
14 public int compareTo(GottRider otherRider);
15
16 // O(1) - returns the average rating this rider gives
17 public Double getAverageReviewGiven();
```